

Project Title

Sam

Charlie

Anusha

Mukund

Project Goal and Scope

The primary objective is to develop a photo-realistic 3D graphics rendering system in C++ utilizing advanced Path Tracing Techniques.

The specific target for the rendering competition is to accurately recreate an image of a chess scene [1]. Achieving this level of visual fidelity requires a synthesis of custom geometric modeling, designing material properties, and a Path Tracer that handles shadows, and glass surfaces.

Accomplishments

Bidirectional Path Tracing

We upgraded the core rendering engine to utilize Bidirectional Path Tracing [2]. In traditional Monte Carlo path tracing, rays are exclusively cast from the camera into the scene. Our implementation improves upon this by simultaneously generating sub-paths from both the camera (eye paths) and the emissive light sources (light paths). The length of both sub-paths is determined stochastically, utilizing Russian Roulette for unbiased termination.

Technical Details

To evaluate the final pixel color, the integrator attempts to connect the vertices of the camera sub-path with the vertices of the light sub-path. For each attempted connection, a deterministic visibility check (shadow ray) is cast between the two endpoints. If an object occludes the line of sight between the vertices, no color contribution is added for that specific connection. If the path is unoccluded, the radiance is evaluated and accumulated similarly to standard Monte Carlo integration.

Path Tracing vs. Bidirectional Path Tracing

In standard unidirectional Path Tracing, the rendering equation is integrated exclusively by gathering light. Rays are cast from the camera into the scene, scattering at surfaces according to the material's BRDF until they randomly intersect an emissive light source or terminate via Russian Roulette. The final color at a surface is evaluated by accumulating the emitted radiance from the final hit point, attenuated by the material reflectance and geometric terms along the reverse path. However, this Monte Carlo integration method often struggles with complex indirect lighting, leading to high variance (noise) because camera paths rarely hit the light source purely by chance.

The rendering equation used to describe traditional path tracing

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (\omega_i \cdot n) d\omega_i \quad (1)$$

The previous equation is evaluated in Recursively. A ray can be traced from the camera eye into the nearest hit point x , then sample a new direction ω_i above x and repeat these steps until some kind of termination condition, such as Russian roulette,

The rendering equation (1) can be transformed into a path integral formulation where instead of integrating over all directions of incoming light over the point p , we integrate over points on the surfaces of objects.

$$L_p = \int_D f_j(\vec{p}) dD \quad (2)$$

where L_p is the radiance that flows through a pixel, D is all possible light paths and $\vec{p}$ is all possible light paths.

This can be approximated using monte-carlo approximation as follows

$$L_p = \frac{1}{N} \sum_{i=0}^N \frac{f_j(\vec{p}_i)}{p(\vec{p}_i)} \quad (3)$$

where $p(\vec{p}_i)$ is the pdf of sampling path $\vec{p}_i$. The conversion into a pdf with respect to surface area $p(x)$ is written as:

$$p(x) = p(\omega) \left(\frac{N_{x'} \cdot x'x''}{\|x'' - x'\|} \right) \quad (4)$$

where $p(\omega)$ is the solid angle. Moreover, We know that

$$L_r(x, \omega_0) = L_{direct} + L_{indirect} \quad (5)$$

(5) can be written with conversion to surface area as

$$L_p = \frac{1}{N} \sum_{i=0}^N L_{p_i} \quad (6)$$

where L_{p_i} is the radiance of a single sample which is calculated by

$$L_{p_i} = \sum_{t=0}^{N_E} C_t \quad (7)$$

Where C_t is the contribution of a single path of length t and N_E is the maximum path length. Applying monte carlo integration on the contribution gives us

$$C_t = \frac{L_e(y_0, \overrightarrow{y_0x_t})}{p(y_0)} f_r(x_t, \overrightarrow{x_t y_0}, \overrightarrow{x_t x_{t-1}}) G(x_t \leftrightarrow y_0) \prod_{i=1}^{t-1} \frac{f_r(x_i, \overrightarrow{x_i x_{i+1}}, \overrightarrow{x_i x_{i-1}}) |N_{x_i} \cdot \overrightarrow{x_i x_{i+1}}|}{p(\overrightarrow{x_i x_{i+1}})} \quad (8)$$

The first part of the equation corresponds to the direct lighting from y_0 onto x_t , where $L_e(y_0, \overrightarrow{y_0x_t})$, is the amount of radiance from point y_0 going in the direction towards x_t . Then it is multiplied with the geometric term, and also with the BRDF. The rest of the equation belongs to indirect lighting. It is computed for each point on the path as the product of BRDF and the dot product, divided by the PDF relative to BRDF.

The geometric coupling term can be defined as

$$G(x \leftrightarrow x') = V(x \leftrightarrow x') \frac{|N_x \cdot \overrightarrow{x'x}| |N_{x'} \cdot \overrightarrow{xx'}|}{\|x - x'\|^2} \quad (9)$$

where $V(x \leftrightarrow x')$ is a visibility function, which is equal to 1 if a ray can be shot directly from point x to x' without hitting anything else, otherwise it is 0

The light tracing which represents a light path can be written using the path integral framework (2) as follows

$$L_p = \sum_{s=0}^{N_L} C_s \frac{\|P_{pixel} - P_{eye}\|^2}{(\cos \theta)^2 A_{pixel}} \quad (10)$$

where the sum represents path, from point y_0 to the maximum path length N_L , C_s is the single path contribution and the rest of the equation is the conversion from flux to radiance. Contribution C_s can be computed as

$$C_s = \frac{L_e(y_0, \overrightarrow{y_0 y_1})}{p(y_0, \overrightarrow{y_0 y_1})} f_r(y_s, \overrightarrow{y_s y_{s+1}}) G(y_s \leftrightarrow x_0) \frac{W_e(x_0, \overrightarrow{y_t x_0})}{p(x_0)} \left(\prod_{i=1}^{s-1} \frac{f_r(y_i, \overrightarrow{y_i y_{i+1}}) |N_{y_i} \cdot \overrightarrow{y_i y_{i+1}}|}{p(\overrightarrow{y_i y_{i+1}})} \right) \quad (11)$$

The first part is the emitted light $L_e(y_0, \overrightarrow{y_0 y_1})$ from the light source at point y_0 . The PDF $p(y_0, \overrightarrow{y_0 y_1})$ is probability of selecting a ray in the direction $\overrightarrow{y_0 y_1}$. Considering a light that has uniform distribution of the emitted light, then probability density function is:

$$p(y_0, \overrightarrow{y_0 y_1}) = \frac{1}{A_{light} 2\pi} \quad (12)$$

y_0 can be sampled over the light area A_{light} in all directions over the hemisphere above the point y_0 . $W_e(x_0, \overrightarrow{y_t x_0})$ is the potential of the light ray going straight from the light source to the pixel, which is equal to 1, when the ray is in front of the camera, and when using a pinhole camera $p(x_0)$ is equal to 1 as well, because only x_0 can be chosen as a point.

The next part $f_r(y_s, \overrightarrow{y_s y_{s+1}}) G(y_s \leftrightarrow x_0)$ represents a direct ray from y_s to the eye x_0 . The third part of the equation is the product of: the BRDF, multiplied with the dot product between normal and direction $\overrightarrow{y_i y_{i+1}}$ and normalized by the PDF.

We have defined both the strategies of ray evaluation, and now Bidirectional path tracing can be defined as

$$L_p = \sum_{s=0}^{N_L} \sum_{t=0}^{N_E} w_{s,t} \cdot C_{s,t} \quad (13)$$

where $C_{s,t}$ is the unweighted contribution of a path with s vertices on the light path and t vertices on the eye path. A visibility check between s^{th} point on the eye path and t^{th} point on the light path is a part of the geometric term in contribution function. And finally, $w_{s,t}$ is the weight function.

As it can be seen, evaluation of the $C_{s,t}$ is independent on the eye and light path; therefore, there are four important cases:

- $s = 0, t = 0$ light is directly visible from the eye, the evaluation can be done with direct path between eye the point x_0 and y_0
- $s > 0, t = 0$ in this case use light tracing defined in (11)
- $s = 0, t > 0$ corresponds to the classic path tracing case defined in (8)
- $s > 0, t > 0$ the final case is the evaluation of radiance from the eye path t connected with s vertices of the light path, reaching the pixel, defined as follows

$$C_{s,t} = \frac{L_e(y_0, \overrightarrow{y_0 y_1})}{p(y_0, \overrightarrow{y_0 y_1})} G(y_s \leftrightarrow x_t) f_r(x_t, \overrightarrow{x_t y_s}, \overrightarrow{x_t x_{t-1}}) f_r(y_s, \overrightarrow{y_s y_{s+1}}, \overrightarrow{y_s x_t}) \\ \times \left(\prod_{i=1}^{t-1} \frac{f_r(x_i, \overrightarrow{x_i x_{i+1}}, \overrightarrow{x_i x_{i-1}}) |N_{x_i} \cdot \overrightarrow{x_i x_{i+1}}|}{p(\overrightarrow{x_i x_{i+1}})} \right) \\ \times \left(\prod_{i=1}^{s-1} \frac{f_r(y_i, \overrightarrow{y_i y_{i+1}}, \overrightarrow{y_i y_{i-1}}) |N_{y_i} \cdot \overrightarrow{y_i y_{i+1}}|}{p(\overrightarrow{y_i y_{i+1}})} \right) \quad (14)$$

The most effective way to use multiple importance sampling in order to create a weight function is with the power heuristic:

$$w_{s,t} = \frac{p_s^\beta}{\sum_{i=0}^{s+t-1} p_i^\beta} \quad (15)$$

where the recommended value for $\beta = 2$

where p_i is defined as the density for generating path $x_{s,t}$ using i sub light path vertices and $s+t-i$ eye sub path vertices :

$$p_i = p_{i,s+t-i}(x_{s,t}^{\rightarrow}) \quad (16)$$

The algorithm suggested in [2] was implemented and is as follows:

Algorithm 1 BDPT

```
1: function RUN(scene) define number of light and eye paths to proceed
2: end function
3: for all lightpaths do
4: end for
```

Rendering Results and Experiments

Discussion

References

- [1] Srinath Ravichandran. Dartmouth rendering competition, 2016.
- [2] Michal Vlnas. Bidirectional path tracing. In *"Proceedings of CESC G 2018: The 22nd Central European Seminar on Computer Graphics"*, 2018.